

Лабораторная работа №33-34. Полноценный CRUD с базой данных

Цель работы:

- Закрепить навыки работы с EF Core на более сложном проекте;
- Освоить валидацию данных через Data Annotations и FluentValidation;
- Реализовать связи между таблицами (один-ко-многим) на уровне С# и БД;
- Научиться обрабатывать ошибки и возвращать понятные сообщения клиенту;
- Применить паттерн Repository для разделения логики доступа к данным и логики контроллера;
- Создать полноценное API, готовое к использованию фронтендом.

Краткая теория (вводная часть)

Что значит «полноценный» CRUD?

В предыдущих лабораторных мы реализовали базовый CRUD. Однако в реальных проектах одних операций «добавить / получить / обновить / удалить» недостаточно. Полноценный CRUD включает:

Аспект	Базовый	Полноценный (эта ЛР)
Валидация	Ручная проверка в контроллере	Data Annotations + централизованная обработка
Ошибки	Разные форматы сообщений	Единый формат ошибок
Связи	Одна таблица	Несколько связанных таблиц
Архитектура	Вся логика в контроллере	Разделение через Repository
Документация	Базовый Swagger	Swagger с описаниями и примерами

Паттерн Repository

Repository — это паттерн проектирования, который изолирует логику работы с данными от логики контроллера.

Без Repository:

Контроллер → напрямую работает с DbContext → База данных

С Repository:

Контроллер → Repository → DbContext → База данных

Зачем это нужно:

Проблема без Repository	Решение с Repository
Логика запросов к БД размазана по контроллерам	Все запросы в одном месте
Трудно тестировать контроллер отдельно от БД	Можно подменить репозиторий на тестовый
При смене БД нужно менять все контроллеры	Меняется только репозиторий
Дублирование одинаковых запросов	Переиспользуемые методы

Аналог из реального мира: представьте склад (база данных) и кладовщика (Repository). Продавцы (контроллеры) не ходят на склад сами — они отправляют запрос кладовщику. Кладовщик знает, где всё лежит и как правильно работать со складом.

Связи между таблицами

В реляционных базах данных таблицы связываются через **внешние ключи** (Foreign Key).

Сегодня реализуем связь **один-ко-многим**:

- Одна **категория** содержит много **задач**
- Одна задача принадлежит ровно одной категории

В C# это выглядит как **навигационные свойства**:

```
// В Task: ссылка на родительскую категорию
public Category Category { get; set; }
public int CategoryId { get; set; }
// В Category: список дочерних задач
public List<Task> Tasks { get; set; }
```

EF Core автоматически создаёт **FOREIGN KEY** в SQLite по этим навигационным свойствам.

Единый формат ошибок

Хорошее API всегда возвращает ошибки в одном предсказуемом формате. Клиент (JavaScript) знает, как их обработать:

```
{
  "success": false,
  "message": "Категория не найдена",
  "errors": ["CategoryId: Категория с id=99 не существует"],
  "statusCode": 404
}
```

И успешный ответ в том же стиле:

```
{
  "success": true,
  "data": { "id": 1, "title": "Моя задача" },
  "message": "Задача успешно создана"
}
```

Шаг 1. Подготовка проекта

В этой лабораторной мы строим приложение «Заметки» — более жизненный пример, чем просто задачи. У нас будут **категории** (Работа, Личное, Учёба) и **заметки** в этих категориях.

1.1. Создание рабочей директории и проекта

1. Откройте любой терминал (например **Git Bash**).
2. Перейдите в каталог с документами.
3. Создайте директорию для лабораторной работы и папку для скриншотов:

```
mkdir Lab33-34_FullCRUD
```

```
cd Lab33-34_FullCRUD
```

```
mkdir img
```

4. Создайте проект **webapi**:

```
dotnet new webapi -n NotesApp
```

5. Откройте проект в VS Code:

```
code .
```

6. Откройте терминал и выполните:

```
cd
```

```
dotnet restore
```

7. Установим **Swagger**. Выполните в терминале:

```
dotnet add package Swashbuckle.AspNetCore
```

```
dotnet restore
```

8. Создайте папки внутри **NotesApp**:

```
mkdir Models Data Repositories Helpers Controllers
```

```
mkdir Models/DTOs
```

1.2. .editorconfig

В корне папки **Lab33-34_FullCRUD** создайте файл **.editorconfig**. Добавьте в ваш .editorconfig файл следующие правила (все скобки на той же строке):

```
[*.cs]
csharp_new_line_before_open_brace = none
```

1.3. Инициализация Git

1. Выполните в терминале:

```
cd ..
```

```
git init -b main
```

```
cd NotesApp
```

```
dotnet new gitignore
```

2. Откройте файл **.gitignore** и добавьте в конец файла:

```
bin/
```

```
obj/
```

3. Выполните в терминале:

```
cd ..
```

```
touch README.md
```

```
git add .
```

```
git commit -m "Initial commit: NotesApp project"
```

4. Создайте **пустой public-репозиторий** на GitHub с именем: **Lab33-34_FullCRUD**

Не добавляйте README, .gitignore, License

5. Привяжите и отправьте:

```
git remote add origin <URL_репозитория>
```

```
git push -u origin main
```

6. Сделайте **скриншот терминала** с выполненными командами (commit, remote add, push) и сохраните в папку img с именем: **gitPushLab33-34_FIO.png**

Реализуйте следующую структуру проекта:

NotesApp/

├── **Controllers/**

| ├── **CategoriesController.cs**

| └── **NotesController.cs**

```
|— Data/
|   |— AppDbContext.cs
|— Helpers/
|   |— ApiResponse.cs      ← единый формат ответов
|— Models/
|   |— Category.cs
|   |— Note.cs
|   |— DTOs/
|       |— CategoryDtos.cs
|       |— NoteFilterDto.cs
|       |— NoteDtos.cs
|— Repositories/
|   |— ICategoryRepository.cs ← интерфейс
|   |— CategoryRepository.cs ← реализация
|   |— INoteRepository.cs
|   |— NoteRepository.cs
|— Program.cs
|— appsettings.json
```

Шаг 2. Установка пакетов и настройка окружения

2.1. Установка NuGet-пакетов

Выполните в терминале:

```
cd NotesApp
```

```
dotnet add package Microsoft.EntityFrameworkCore
```

```
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
```

```
dotnet add package Microsoft.EntityFrameworkCore.Tools
```

```
dotnet add package Swashbuckle.AspNetCore
```

Swashbuckle.AspNetCore — это и есть Swagger для ASP.NET Core.

Проверьте dotnet-ef:

```
dotnet ef --version
```

Если не установлен:

```
dotnet tool install --global dotnet-ef
```

2.2. Настройка appsettings.json

Откройте **appsettings.json** и добавьте строку подключения:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "NotesDb": "Data Source=notesapp.db"
  }
}
```

Добавьте логирование SQL в **appsettings.Development.json**:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning",
      "Microsoft.EntityFrameworkCore.Database.Command": "Information"
    }
  }
}
```

Практическое задание

Выполните в терминале:

dotnet build

Убедитесь, что сборка прошла без ошибок.

cd ..

git add .

git commit -m "Step 2: packages installed, appsettings configured"

cd NotesApp

Шаг 3. Модели и связи между таблицами

3.1. Модель Category

Откройте файл **Models/Category.cs** и вручную перепишите код.

1. Объявляем пространства имён

```
using System.ComponentModel.DataAnnotations;
namespace NotesApp.Models;
```

using System.ComponentModel.DataAnnotations; - Подключаем пространство имён, в котором хранятся атрибуты валидации: [Required], [MaxLength], [Range] и другие. Без этой строки компилятор не найдёт эти атрибуты.

namespace NotesApp.Models; - Помещаем класс в пространство имён **NotesApp.Models**. Это как папка для файлов: все модели лежат в одном «логическом пространстве». Когда другой файл пишет **using NotesApp.Models;** — он получает доступ к этому классу.

2. Объявляем публичный класс **Category**

```
public class Category { }
```

EF Core увидит этот класс и создаст в базе данных таблицу с именем **Categories** (автоматически добавляет «s» во множественном числе).

3. Далее объявляем **внутри** класса **поля**

```
public int Id { get; set; }
public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
public List<Note> Notes { get; set; } = new();
```

Id - Первичный ключ таблицы. **EF Core** по соглашению считает свойство с именем **Id** (или **CategoryId**) первичным ключом и делает его **PRIMARY KEY AUTOINCREMENT** в **SQLite**.

CreatedAt - Дата и время создания записи. **DateTime.UtcNow** возвращает текущее время в формате UTC (всемирное координированное время). UTC используется вместо локального времени, чтобы приложение работало одинаково на серверах в разных часовых поясах.

Notes - Навигационное свойство — ключевая концепция **EF Core**. Это свойство не становится колонкой в таблице **Categories**. Вместо этого **EF Core** использует его, чтобы понять связь между таблицами:

- Видит **List<Note>** в классе **Category**
- Видит **int CategoryId** в классе **Note**
- Автоматически создаёт **FOREIGN KEY (CategoryId) REFERENCES Categories(Id)** в таблице **Notes**

= new() — сокращённая запись = new List<Note>(). Инициализируем пустым списком, чтобы при обращении к category.Notes не возникало NullReferenceException, если заметки ещё не загружены.

```
[Required(ErrorMessage = "Название категории  
обязательно")]  
[MaxLength(100, ErrorMessage = "Название не должно  
превышать 100 символов")]  
public string Name { get; set; } = string.Empty;
```

Три строки — одно свойство с двумя атрибутами валидации:

- **[Required]** — поле обязательно для заполнения. Если клиент пришлёт запрос без поля Name, ASP.NET Core автоматически вернёт ошибку 400 Bad Request с текстом из ErrorMessage.
- **[MaxLength(100)]** — ограничение длины строки. EF Core создаст в БД колонку VARCHAR(100), а ASP.NET Core будет проверять длину до сохранения.
- = **string.Empty** — инициализируем свойство пустой строкой. Это важно: без инициализации компилятор C# 10+ предупредит, что строковое свойство может быть null. string.Empty — это то же самое, что "", но более явно.

```
[MaxLength(500, ErrorMessage = "Описание не должно  
превышать 500 символов")]  
public string Description { get; set; } = string.Empty;
```

Описание категории — необязательное поле (нет **[Required]**), но с ограничением длины. Если клиент не передаст **Description**, поле просто останется пустой строкой.

```
[MaxLength(7)]  
public string Color { get; set; } = "■#3498db";
```

HEX-цвет в формате **#RRGGBB**. Длина ровно 7 символов: # + 6 шестнадцатеричных цифр. Значение по умолчанию — синий цвет. Этот цвет будет использоваться фронтендом для визуального отображения категории.

3.2. Модель Note

Откройте файл **Models/Note.cs** и вручную перепишите код.

1. Объявляем пространства имён


```
using System.ComponentModel.DataAnnotations;
namespace NotesApp.Models;
```

2. Объявляем публичный класс **Note**

```
public class Note { }
```

3. Далее объявляем **внутри** класса поля

```
public int Id { get; set; }
public int CategoryId { get; set; }
public Category Category { get; set; } = null!;
public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
public DateTime UpdatedAt { get; set; } = DateTime.UtcNow;
public bool IsPinned { get; set; } = false;
public bool IsArchived { get; set; } = false;
```

CategoryId. Внешний ключ (Foreign Key). Это обычная целочисленная колонка в таблице Notes. Хранит Id той категории, к которой принадлежит заметка. Например, если категория «Работа» имеет Id = 2, то все рабочие заметки будут иметь CategoryId = 2.

EF Core распознаёт это свойство автоматически по соглашению: имя навигационного свойства (Category) + Id = CategoryId. В базе данных это создаст:

```
CategoryId INTEGER NOT NULL,
FOREIGN KEY (CategoryId) REFERENCES Categories(Id)
```

Category. Навигационное свойство для загрузки связанного объекта.

Если CategoryId — это просто число 2, то Category — это полный объект категории с именем, цветом и описанием. EF Core заполняет это свойство автоматически, когда вы делаете `.Include(n => n.Category)` в запросе.

`= null!` — **оператор null-прощения** (null-forgiving operator). Разберём подробнее:

- Технически, до загрузки из БД это свойство равно `null`
- Компилятор C# 10+ об этом предупреждает: «свойство не может быть `null`»
- `null!` означает: «компилятор, я знаю, что это `null` сейчас, но EF Core заполнит это поле — не показывай предупреждение»
- Это безопаснее, чем `= new Category()`, потому что не создаёт фиктивный пустой объект

 **null!** используется **только** для навигационных свойств EF Core. В обычном коде использовать его не нужно.

Свойство	Тип в C#	Колонка в SQLite	Роль
Id	int	Id INTEGER PRIMARY KEY	Первичный ключ заметки
CategoryId	int	CategoryId INTEGER NOT NULL	Внешний ключ
Category	Category	(нет колонки)	Навигационное свойство

CreatedAt - устанавливается один раз при создании заметки и больше не меняется.

UpdatedAt - обновляется каждый раз, когда заметка редактируется — это нужно делать вручную в коде репозитория (в шаге 5). Оба изначально равны текущему времени UTC.

IsPinned и **IsArchived** - Булевы флаги. В SQLite хранятся как 0 (false) или 1 (true). EF Core преобразует их автоматически. **IsPinned** — закреплённая заметка (показывается вверху списка), **IsArchived** — архивная (скрыта из основного списка). По умолчанию оба false.

```
[Required(ErrorMessage = "Заголовок заметки обязателен")]
[MaxLength(200, ErrorMessage = "Заголовок не должен превышать 200 символов")]
public string Title { get; set; } = string.Empty;
```

```
[MaxLength(5000, ErrorMessage = "Содержимое не должно превышать 5000 символов")]
public string Content { get; set; } = string.Empty;
```

```
[Range(1, 5, ErrorMessage = "Приоритет должен быть от 1 до 5")]
public int Priority { get; set; } = 3;
```

Атрибут **[Range(1, 5)]** проверяет, что значение находится в диапазоне от 1 до 5 включительно. Если клиент пришлёт **Priority**: 10 — ASP.NET Core вернёт ошибку валидации автоматически, не дойдя до кода контроллера. Значение по умолчанию 3 — средний приоритет.

3.3. Модели DTO

DTO (Data Transfer Object) — объект для передачи данных между клиентом и сервером. Это отдельный класс, который описывает только те поля, которые нужны в конкретной операции.

Без DTO	С DTO
Клиент видит все поля модели, включая служебные	Клиент видит только нужные поля

При сериализации возникает рекурсия (Category → Плоская структура без рекурсии Notes → Category → ...)
Нельзя разделить правила валидации для создания и обновления Разные правила для Create и Update

Откройте файл **Models/DTOs/CategoryDtos.cs** и вручную перепишите код.

1. Объявляем пространства имён

```
using System.ComponentModel.DataAnnotations;
namespace NotesApp.Models.DTOs;
```

DTO-классы размещаем в подпространстве **DTOs** — это отражает структуру папок (*Models/DTOs/*). Такое разделение помогает сразу понять: это не сущность базы данных, а объект для передачи данных.

2. Создаём класс для создания категории **CreateCategoryDto**

```
public class CreateCategoryDto {
    [Required(ErrorMessage = "Название обязательно")]
    [MaxLength(100, ErrorMessage = "Максимум 100 символов")]
    public string Name { get; set; } = string.Empty;

    [MaxLength(500)]
    public string Description { get; set; } = string.Empty;

    [MaxLength(7)]
    [RegularExpression(@"^[0-9A-Fa-f]{6}$",
        ErrorMessage = "Цвет должен быть в формате HEX: #RRGGBB")]
    public string Color { get; set; } = "#3498db";
}
```

[RegularExpression] проверяет строку по регулярному выражению. Разберём паттерн `^[0-9A-Fa-f]{6}$`:

Часть	Значение
<code>^</code>	Начало строки
<code>#</code>	Символ # обязателен
<code>[0-9A-Fa-f]</code>	Любая шестнадцатеричная цифра: 0-9, a-f, A-F
<code>{6}</code>	Ровно 6 таких символов
<code>\$</code>	Конец строки

Примеры: `#3498db` , `#FF0000` , `red` , `#GGG` 

`@` перед строкой — **verbatim string** (дословная строка). Позволяет не экранировать обратный слеш `\`, что удобно для регулярных выражений.

3. Создаём класс для обновления категории **UpdateCategoryDto**

```

public class UpdateCategoryDto {
    [Required(ErrorMessage = "Название обязательно")]
    [MaxLength(100, ErrorMessage = "Максимум 100 символов")]
    public string Name { get; set; } = string.Empty;

    [MaxLength(500)]
    public string Description { get; set; } = string.Empty;

    [MaxLength(7)]
    [RegularExpression(@"^[0-9A-Fa-f]{6}$",
        ErrorMessage = "Цвет должен быть в формате HEX: #RRGGBB")]
    public string Color { get; set; } = "■#3498db";
}

```

UpdateCategoryDto выглядит идентично **CreateCategoryDto**. Это намеренно: сейчас правила одинаковые, но в будущем они могут разойтись. Например, при обновлении может появиться поле **UpdatedBy** (кто обновил), которого нет при создании. Разделение с самого начала — хорошая практика.

4. Создаём класс для ответа (включает количество заметок, без самих заметок)

CategoryResponseDto

```

public class CategoryResponseDto {
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public string Description { get; set; } = string.Empty;
    public string Color { get; set; } = string.Empty;
    public DateTime CreatedAt { get; set; }
    public int NotesCount { get; set; }
}

```

CategoryResponseDto — это то, что сервер **возвращает** клиенту.

Обратите внимание: здесь **нет** атрибутов **[Required]** и **[MaxLength]** — они нужны только для входящих данных, но не для исходящих ответов.

NotesCount — **вычисляемое поле**: количество заметок в категории. Это поле отсутствует в модели **Category**, но будет вычисляться в репозитории через **.Count()**. Клиенту не нужно получать весь список заметок, чтобы узнать их количество.

Также здесь **нет** **List<Note> Notes** — это сделано намеренно, чтобы избежать бесконечной рекурсии при сериализации в JSON.

Откройте файл **Models/DTOs/NoteDtos.cs** и вручную перепишите код.

1. Объявляем пространства имён

```
using System.ComponentModel.DataAnnotations;  
namespace NotesApp.Models.DTOS;
```

2. Создаём класс для создания заметок **CreateNoteDto**

```
public class CreateNoteDto {  
    [Required(ErrorMessage = "Заголовок обязателен")]  
    [MaxLength(200, ErrorMessage = "Максимум 200 символов")]  
    public string Title { get; set; } = string.Empty;  
  
    [MaxLength(5000, ErrorMessage = "Максимум 5000 символов")]  
    public string Content { get; set; } = string.Empty;  
  
    [Range(1, 5, ErrorMessage = "Приоритет от 1 до 5")]  
    public int Priority { get; set; } = 3;  
  
    [Required(ErrorMessage = "Категория обязательна")]  
    public int CategoryId { get; set; }  
}
```

При создании заметки клиент обязан указать **CategoryId**. **[Required]** на **int** работает немного иначе, чем на **string**: он проверяет, что значение не равно дефолтному 0 (несуществующий Id). Если клиент не передаст это поле — получит ошибку валидации. Обратите внимание: в **CreateNoteDto** нет полей **IsPinned**, **IsArchived**, **CreatedAt**, **UpdatedAt** — клиент не должен их задавать. Они устанавливаются сервером автоматически.

3. Создаём класс для обновления заметок **UpdateNoteDto**

```
public class UpdateNoteDto {  
    [Required(ErrorMessage = "Заголовок обязателен")]  
    [MaxLength(200, ErrorMessage = "Максимум 200 символов")]  
    public string Title { get; set; } = string.Empty;  
  
    [MaxLength(5000, ErrorMessage = "Максимум 5000 символов")]  
    public string Content { get; set; } = string.Empty;  
  
    [Range(1, 5, ErrorMessage = "Приоритет от 1 до 5")]  
    public int Priority { get; set; } = 3;  
  
    [Required(ErrorMessage = "Категория обязательна")]  
    public int CategoryId { get; set; }  
}
```

4. Создаём класс ответа (включает название категории) **NoteResponseDto**

```

public class NoteResponseDto {
    public int Id { get; set; }
    public string Title { get; set; } = string.Empty;
    public string Content { get; set; } = string.Empty;
    public bool IsPinned { get; set; }
    public bool IsArchived { get; set; }
    public int Priority { get; set; }
    public DateTime CreatedAt { get; set; }
    public DateTime UpdatedAt { get; set; }
    public int CategoryId { get; set; }
    public string CategoryName { get; set; } = string.Empty;
    public string CategoryColor { get; set; } = string.Empty;
}

```

CategoryId и **CategoryName**. Эти два поля приходят из связанной таблицы **Categories**. В базе данных они не хранятся в таблице **Notes** — они будут получены через **JOIN** (в EF Core это `.Include()`).

Итоговая карта DTO

Клиент → Сервер: <code>CreateNoteDto / UpdateNoteDto</code>	(входящие данные + валидация)
Сервер → Клиент: <code>NoteResponseDto</code>	(исходящий ответ, плоская структура)
Внутри сервера: <code>Note</code> (модель EF Core)	(хранение в БД, навигационные свойства)

Практическое задание

Выполните в терминале:

dotnet build

Сборка должна пройти без ошибок. Далее:

cd ..

git add .

git commit -m "Step 3: Category and Note models with relationships and DTOs"

cd NotesApp

Шаг 4. Helper-классы и DbContext

4.1. Единый формат ответа

Откройте файл **Helpers/ApiResponse.cs** и вручную перепишите код.

1. Объявляем пространства имён

```
namespace NotesApp.Helpers;
```

2. Создаём класс обёртку для успешных ответов с данными **ApiResponse**


```

public class ApiResponse<T> {
    public bool Success { get; set; }
    public string Message { get; set; } = string.Empty;
    public T? Data { get; set; }
    public int StatusCode { get; set; }

    public static ApiResponse<T> Ok(T data, string message = "Успешно")
        => new() { Success = true, Data = data, Message = message, StatusCode = 200 };

    public static ApiResponse<T> Created(T data, string message = "Создано успешно")
        => new() { Success = true, Data = data, Message = message, StatusCode = 201 };
}

```

<T> — **обобщённый тип (generic)**. Это та же концепция, что `List<string>` или `List<int>` — вместо `T` подставляется любой конкретный тип. Благодаря этому один класс работает для любых данных:

`ApiResponse<CategoryResponseDto>` // ответ с одной категорией

`ApiResponse<List<NoteResponseDto>>` // ответ со списком заметок

`ApiResponse<string>` // ответ с текстовым сообщением

Без generic пришлось бы писать отдельный класс для каждого типа данных.

Четыре поля, которые будут в **каждом** ответе API:

Поле	Тип	Назначение
Success	bool	Всегда true в ApiResponse — операция прошла успешно
Message	string	Человекочитаемое сообщение: «Успешно», «Создано успешно»
Data	T?	Сами данные — категория, заметка, список и т. д.
StatusCode	int	HTTP-статус: 200, 201, 404, 400, 500

T? — знак вопроса означает, что данные могут быть **null**. Это важно: в ответах об ошибках данных нет.

static ApiResponse<T> Ok

- **public static** — метод вызывается без создания объекта: `ApiResponse<T>.Ok(...)`. Это фабричный метод — он создаёт и возвращает готовый объект.
- **ApiResponse<T>** — тип возвращаемого значения.
- **string message = "Успешно"** — параметр со значением по умолчанию. Если при вызове не передать `message`, автоматически подставится "Успешно".
- **=> new() { ... }** — стрелочная запись метода (expression body). Это короткий способ написать метод из одной строки.

- **new()** { ... } — создание объекта с инициализатором. **new()** без имени типа работает, потому что C# сам понимает, что создаётся **ApiResponse<T>** (тип уже указан в сигнатуре метода).

static ApiResponse<T> Created. Аналогичен **Ok**, но со статусом 201 **Created**. По стандарту REST, 201 возвращается при успешном создании нового ресурса (POST-запрос). 200 — для чтения и обновления.

3. Создаём класс обёртку для ответов об ошибках **ApiError**

```
public class ApiError {
    public bool Success { get; set; } = false;
    public string Message { get; set; } = string.Empty;
    public List<string> Errors { get; set; } = new();
    public int StatusCode { get; set; }

    public static ApiError NotFound(string message)
        => new() { Message = message, StatusCode = 404 };

    public static ApiError BadRequest(string message, List<string>? errors = null)
        => new() { Message = message, Errors = errors ?? new(), StatusCode = 400 };

    public static ApiError Internal(string message = "Внутренняя ошибка сервера")
        => new() { Message = message, StatusCode = 500 };
}
```

Отдельный класс для ошибок (не generic — в ошибках данных нет). **Success = false** задан прямо в объявлении свойства, чтобы не забыть установить его в каждом фабричном методе.

List<string> Errors - Список конкретных ошибок валидации. Нужен, когда одновременно несколько полей не прошли проверку.

static ApiError NotFound - Фабричный метод для ошибки «не найдено». Используется когда ищем запись по **Id**, а её нет в базе.

static ApiError BadRequest - *List<string>? errors = null* — необязательный параметр. Можно вызвать с ошибками или без. **errors ?? new()** — оператор ?? (null-coalescing). Означает: «если **errors** равен **null**, используй **new List<string>()** (пустой список)». Это гарантирует, что поле **Errors** никогда не будет **null** в ответе — всегда хотя бы пустой массив **[]**.

static ApiError Internal - Для неожиданных ошибок сервера (исключения, сбои БД). Статус **500** означает, что проблема на стороне сервера, а не в запросе клиента. Параметр со значением по умолчанию позволяет вызывать как **ApiError.Internal()** без аргументов.

Теперь все ответы будут выглядеть одинаково. Клиент (JavaScript) может положиться на поля **success**, **data**, **message**.

4.2. DbContext со связями и начальными данными

Откройте файл **Data/AppDbContext.cs** и вручную перепишите код.

1. Объявляем пространства имён

```
using Microsoft.EntityFrameworkCore;
using NotesApp.Models;
namespace NotesApp.Data;
```

2. Объявляем класс AppDbContext

```
public class AppDbContext : DbContext {
    public AppDbContext(DbContextOptions<AppDbContext>
options) : base(options) { }
}
```

AppDbContext наследует от **DbContext** — базового класса EF Core. **DbContext** содержит всю логику взаимодействия с базой данных: открытие соединений, отслеживание изменений объектов, выполнение SQL-запросов. Наш класс только добавляет к нему конкретные таблицы и настройки.

public AppDbContext - Конструктор класса. Разберём по частям:

- **DbContextOptions<AppDbContext> options** — объект с настройками: строка подключения к БД, тип базы данных (SQLite, PostgreSQL и др.), параметры логирования. Эти настройки задаются в **Program.cs**.
- **: base(options)** — передаём настройки в конструктор родительского класса **DbContext**. Это обязательно — без этого EF Core не будет знать, к какой базе подключаться.
- **{ }** — тело конструктора пустое, потому что вся работа выполняется в **base(options)**.

3. Внутри класса AppDbContext объявляем поля:

```
public DbSet<Category> Categories { get; set; }
public DbSet<Note> Notes { get; set; }
```

DbSet<T> — представляет таблицу в базе данных. Через это свойство выполняются все запросы:

```
_context.Categories.ToList() // SELECT * FROM Categories
```

```
_context.Notes.Where(n => n.IsPinned) // SELECT * FROM Notes WHERE IsPinned = 1
_context.Categories.Add(newCategory) // INSERT INTO Categories ...
```

Имена свойств (**Categories**, **Notes**) становятся именами таблиц в SQLite.

4. **Внутри** класса **AppDbContext** переопределяем метод **OnModelCreating**:

```
protected override void OnModelCreating(ModelBuilder modelBuilder) {
    base.OnModelCreating(modelBuilder);
}
```

- **OnModelCreating** — метод, который EF Core вызывает **один раз** при запуске приложения, чтобы настроить схему базы данных. Здесь описываются связи, индексы и начальные данные.
- **override** — переопределяем метод родительского класса **DbContext** (уже знакомо из ООП в C#).
- **base.OnModelCreating(modelBuilder)** — сначала вызываем оригинальный метод родителя, чтобы не потерять его стандартную логику, и только потом добавляем свои настройки.

5. Далее пишем **внутри** метода **OnModelCreating**, после **base**. Настраиваем связь **Category (1) —> Notes (много)**:

```
modelBuilder.Entity<Note>()
    .HasOne(n => n.Category) // Note имеет одну Category
    .WithMany(c => c.Notes) // Category имеет много Notes
    .HasForeignKey(n => n.CategoryId) // внешний ключ — CategoryId
    .OnDelete(DeleteBehavior.Restrict); // запрет удаления категории с заметками
```

Это цепочка методов (Fluent API), каждый из которых добавляет уточнение:

Метод	Что говорит EF Core
.Entity<Note>()	«Настраиваем сущность Note»
.HasOne(n => n.Category)	«Каждая заметка имеет одну категорию»
.WithMany(c => c.Notes)	«Каждая категория имеет много заметок»
.HasForeignKey(n => n.CategoryId)	«Внешний ключ — свойство CategoryId»
.OnDelete(DeleteBehavior.Restrict)	«При удалении категории — выдать ошибку, если есть заметки»

Зачем писать явно, если EF Core и так обнаружит связь по навигационным свойствам? Явная настройка нужна для **OnDelete** — по умолчанию EF Core выбирает **Cascade**, что опасно.

n => n.Category и **c => c.Notes** — **лямбда-выражения**. **n** — это переменная типа **Note**, **c** — типа **Category**. Стрелка **=>** означает «для данного объекта взять это свойство».

.OnDelete(DeleteBehavior.Restrict); - Определяет поведение при удалении родительской записи (категории):

Значение	Поведение	Когда использовать
Restrict	Запрет удаления, если есть связанные записи	Когда потеря данных недопустима
Cascade	Удалить категорию → автоматически удалить все её заметки	Когда дочерние записи без смысла без родителя
SetNull	Установить <code>CategoryId = NULL</code> у заметок	Когда заметки могут существовать без категории

Для нашего приложения **Restrict** — правильный выбор: пользователь не потеряет заметки при случайном удалении категории.

6. Внутри метода **OnModelCreating** добавляем индексы для ускорения поиска

```
modelBuilder.Entity<Note>()  
    .HasIndex(n => n.CategoryId);
```

ускоряет поиск заметок по категории

```
modelBuilder.Entity<Note>()  
    .HasIndex(n => n.CreatedAt);
```

ускоряет сортировку по дате

Индексы ускоряют поиск в базе данных. Аналогия: индекс в конце учебника — вместо просмотра всех страниц вы сразу находите нужную.

Без индекса запрос «найди все заметки категории 2» просматривает **все** строки таблицы. С индексом по `CategoryId` — находит нужные записи мгновенно.

HasIndex(n => n.CategoryId) создаёт в SQLite:

```
CREATE INDEX IX_Notes_CategoryId ON Notes (CategoryId);
```

Индекс по **CreatedAt** ускоряет сортировку заметок по дате — частая операция в нашем API.

7. Внутри метода **OnModelCreating** добавляем начальные данные

```
modelBuilder.Entity<Category>().HasData(  
    new Category {  
        Id = 1,  
        Name = "Работа",  
        Description = "Рабочие заметки и задачи",  
        Color = "■ #e74c3c",  
        CreatedAt = new DateTime(2026, 1, 1, 0, 0, 0, DateTimeKind.Utc)  
    },  
);
```

```

new Category {
    Id = 2,
    Name = "Учёба",
    Description = "Конспекты и материалы для обучения",
    Color = "■#3498db",
    CreatedAt = new DateTime(2026, 1, 1, 0, 0, 0, DateTimeKind.Utc)
},
new Category {
    Id = 3,
    Name = "Личное",
    Description = "Личные заметки и идеи",
    Color = "■#2ecc71",
    CreatedAt = new DateTime(2026, 1, 1, 0, 0, 0, DateTimeKind.Utc)
}
);

```

HasData — начальные данные (seed data). Эти записи будут автоматически добавлены в базу при первом запуске миграции. Благодаря этому база не будет пустой сразу после запуска — удобно для разработки и тестирования.

Важные требования к seed data:

- **Id** должен быть задан **явно** — EF Core не может использовать автоинкремент для начальных данных.
- **CreatedAt** задаётся через `new DateTime` (год, месяц, день, час, мин, сек, **DateTimeKind.Utc**), а не **DateTime.UtcNow**. Если использовать **DateTime.UtcNow**, EF Core будет видеть новое значение при каждом запуске и создавать лишние миграции, считая, что данные изменились.

`new DateTime(2026, 1, 1, 0, 0, 0, DateTimeKind.Utc)`

Аргумент	Значение
2026, 1, 1 0, 0, 0	1 января 2026 года 00:00:00 (полночь)
DateTimeKind.Utc	Часовой пояс UTC

8. Внутри метода **OnModelCreating** добавляем начальные данные

```

modelBuilder.Entity<Note>().HasData(
    new Note {
        Id = 1,
        Title = "Изучить ASP.NET Core",
        Content = "Пройти лабораторные работы 27-36.",
        Priority = 5,
        IsPinned = true,
        IsArchived = false,
        CategoryId = 2,
        CreatedAt = new DateTime(2026, 1, 1, 0, 0, 0, DateTimeKind.Utc),
        UpdatedAt = new DateTime(2026, 1, 1, 0, 0, 0, DateTimeKind.Utc)
    },
);

```

```

new Note {
    Id = 2,
    Title = "Подготовить отчёт",
    Content = "Написать отчёт по лабораторной работе №30 до конца недели.",
    Priority = 4,
    IsPinned = false,
    IsArchived = false,
    CategoryId = 1,
    CreatedAt = new DateTime(2026, 1, 2, 0, 0, 0, DateTimeKind.Utc),
    UpdatedAt = new DateTime(2026, 1, 2, 0, 0, 0, DateTimeKind.Utc)
},
new Note {
    Id = 3,
    Title = "Идеи для проекта",
    Content = "Сделать приложение для отслеживания расходов.",
    Priority = 3,
    IsPinned = false,
    IsArchived = false,
    CategoryId = 3,
    CreatedAt = new DateTime(2026, 1, 3, 0, 0, 0, DateTimeKind.Utc),
    UpdatedAt = new DateTime(2026, 1, 3, 0, 0, 0, DateTimeKind.Utc)
},

```

```

new Note {
    Id = 4,
    Title = "Конспект по LINQ",
    Content = "Основные методы для работы с коллекциями и EF Core.",
    Priority = 4,
    IsPinned = true,
    IsArchived = false,
    CategoryId = 2,
    CreatedAt = new DateTime(2026, 1, 4, 0, 0, 0, DateTimeKind.Utc),
    UpdatedAt = new DateTime(2026, 1, 4, 0, 0, 0, DateTimeKind.Utc)
},
new Note {
    Id = 5,
    Title = "Старый черновик",
    Content = "Этот черновик больше не актуален, отправлен в архив.",
    Priority = 1,
    IsPinned = false,
    IsArchived = true,
    CategoryId = 3,
    CreatedAt = new DateTime(2026, 1, 5, 0, 0, 0, DateTimeKind.Utc),
    UpdatedAt = new DateTime(2026, 1, 5, 0, 0, 0, DateTimeKind.Utc)
}
);

```

При заполнении начальных данных для **Note** нельзя использовать навигационное свойство **Category = new Category { ... }**. Нужно указывать только **CategoryId** — числовой внешний ключ. EF Core сам выстроит связь при загрузке данных.

Итоговая схема работы классов

HTTP-запрос



Контроллер

| использует



ApiResponse<T> / ApiError ← единый формат всех ответов



| данные берёт из



Repository

| работает через



AppDbContext

| описывает таблицы

└─ DbSet<Category> → таблица Categories в SQLite

└─ DbSet<Note> → таблица Notes в SQLite

└─ FOREIGN KEY CategoryId → Categories(Id)

Практическое задание

Выполните в терминале:

dotnet build

cd ..

git add .

git commit -m "Step 4: AppDbContext with relationships, indexes, seed data"

cd NotesApp

Шаг 5. Паттерн Repository

5.1. Интерфейс ICategoryRepository

Откройте файл **Repositories/ICategoryRepository.cs** и вручную перепишите код.

1. Объявляем пространства имён

```
using NotesApp.Models;  
using NotesApp.Models.DTOS;  
namespace NotesApp.Repositories;
```

2. Объявляем интерфейс IcategoryRepository

```
public interface ICategoryRepository {
    Task<IEnumerable<CategoryResponseDto>> GetAllAsync();
    Task<Category?> GetByIdAsync(int id);
    Task<Category?> GetByIdWithNotesAsync(int id);
    Task<Category> CreateAsync(Category category);
    Task<Category> UpdateAsync(Category category);
    Task DeleteAsync(Category category);
    Task<bool> ExistsAsync(int id);
    Task<bool> HasNotesAsync(int id);
}
```

interface — описывает **контракт**: список методов, которые обязана реализовать любая имплементация. Интерфейс не содержит кода — только сигнатуры методов. Буква **I** в начале имени — общепринятое соглашение в C# для интерфейсов.

Контроллер будет работать именно с **ICategoryRepository**, а не с конкретным классом. Это даёт два преимущества:

- В тестах можно подменить реальный репозиторий на фиктивный (без обращения к БД)
- При смене базы данных нужно написать новую реализацию интерфейса, не трогая контроллер

Task<IEnumerable<CategoryResponseDto>> GetAllAsync();

Разберём тип возвращаемого значения по частям:

Часть	Значение
Task<...>	Метод асинхронный — не блокирует поток во время ожидания ответа от БД
IEnumerable<...>	Коллекция элементов, которую можно перебрать в foreach
CategoryResponseDto	Тип элемента — DTO для ответа (не сама модель Category)

Суффикс **Async** — соглашение в C#: все асинхронные методы называются с **Async** в конце, чтобы сразу было понятно, что нужен await.

Task<Category?> GetByIdAsync(int id);

Task<Category?> GetByIdWithNotesAsync(int id);

Category? — метод **может вернуть null**, если запись не найдена. Знак **?** обязывает вызывающий код проверять результат на **null**, иначе компилятор выдаст предупреждение.

Два похожих метода с разным назначением:

- **GetByIdAsync** — только данные категории, без заметок. Используется для проверки существования и операций обновления/удаления.
- **GetByIdWithNotesAsync** — категория вместе со всеми заметками через JOIN. Используется только там, где нужны сами заметки — чтобы не грузить лишнее из БД.

Task<bool> ExistsAsync(int id);

Task<bool> HasNotesAsync(int id);

Два специальных метода-проверки:

- **ExistsAsync** — проверяем, существует ли категория с таким Id перед обновлением.
- **HasNotesAsync** — проверяем, есть ли заметки в категории **перед удалением**. Если есть — вернём понятную ошибку вместо технического исключения от **Restrict**.

Возвращают **bool**, а не объект — это эффективнее: в SQL генерируется **SELECT EXISTS(...)** вместо **SELECT ***.

5.2. Реализация **CategoryRepository**

Откройте файл **Repositories/CategoryRepository.cs** и вручную перепишите код.

1. Объявляем пространства имён

```
using Microsoft.EntityFrameworkCore;
using NotesApp.Data;
using NotesApp.Models;
using NotesApp.Models.DTOs;
namespace NotesApp.Repositories;
```

2. Объявляем класс **CategoryRepository**

```
public class CategoryRepository : ICategoryRepository { }
```

: ICategoryRepository — класс берёт на себя обязательство реализовать **все** методы интерфейса. Если хотя бы один метод не реализован — ошибка компиляции. Это гарантия: любой объект типа **ICategoryRepository** точно имеет все нужные методы.

3. Внутри класса добавляем внедрение зависимости (**Dependency Injection**):

```
private readonly AppDbContext _db;
public CategoryRepository(AppDbContext db) {
    _db = db;
}
```

Репозиторий не создаёт **AppDbContext** сам — получает его снаружи через конструктор.

- **private** — поле доступно только внутри этого класса.
- **readonly** — поле можно присвоить только в конструкторе. После этого его нельзя заменить другим объектом. Это защита от случайной ошибки.
- **_db** — соглашение об именовании: приватные поля начинаются с подчёркивания.

ASP.NET Core сам создаст **AppDbContext** и передаст его в конструктор — это называется DI-контейнер (настраивается в **Program.cs** на следующем шаге).

4. **Внутри** класса **CategoryRepository** добавляем метод **GetAllAsync()**, для получения всех категорий с количеством заметок

```
public async Task<IEnumerable<CategoryResponseDto>> GetAllAsync() {
    return await _db.Categories
        .Select(c => new CategoryResponseDto {
            Id = c.Id,
            Name = c.Name,
            Description = c.Description,
            Color = c.Color,
            CreatedAt = c.CreatedAt,
            NotesCount = c.Notes.Count(n => !n.IsArchived)
        })
        .OrderBy(c => c.Name)
        .ToListAsync();
}
```

- **await** приостанавливает выполнение метода до получения результата от базы данных, не блокируя поток. Пока идёт запрос к БД, сервер может обрабатывать другие запросы. Без **await** поток был бы занят ожиданием.
- **.Select()** — проецирует каждую запись в новый тип. EF Core транслирует это в SQL
- Подсчёт **NotesCount** происходит прямо в SQL, а не в C#. Альтернатива — загрузить все заметки в память и посчитать там — намного хуже для производительности при большом количестве данных.
- **!n.IsArchived** — **!** означает «не». Считаем только заметки, у которых **IsArchived == false**.
- **.OrderBy()** — сортировка по имени (А→Я). Транслируется в **ORDER BY Name ASC**.

- **.ToListAsync()** — выполняет запрос и загружает результаты в список. До этого момента SQL-запрос ещё не отправлен в базу — EF Core строит дерево запроса.
- **.ToListAsync()** — это триггер выполнения.

5. Внутри класса **CategoryRepository** добавляем **асинхронные** методы

5.1. **GetByIdAsync()**. Получить категорию без заметок (для проверки существования)

```
public async Task<Category?> GetByIdAsync(int id)
    => await _db.Categories.FindAsync(id);
```

.FindAsync(id) — поиск по первичному ключу. Это специальный метод EF Core: он сначала смотрит в кэш уже загруженных объектов, и только если не находит — идёт в базу. Работает быстрее, чем **.FirstOrDefaultAsync(c => c.Id == id)**.

Возвращает **null**, если запись не найдена — поэтому тип **Category?**.

5.2. **GetByIdWithNotesAsync()**. Получить категорию вместе со всеми заметками (для детального просмотра)

```
public async Task<Category?> GetByIdWithNotesAsync(int id) {
    return await _db.Categories
        .Include(c => c.Notes.Where(n => !n.IsArchived))
        .FirstOrDefaultAsync(c => c.Id == id);
}
```

.Include() — говорит EF Core: «выполни **JOIN** и загрузи связанные объекты». Без **Include** свойство **category.Notes** было бы пустым списком. **Where** внутри **Include** — фильтрованная загрузка (**filtered include**). Загружаем только неархивные заметки.

.FirstOrDefaultAsync() — вернуть первый подходящий элемент или **null**, если ничего не найдено. **First** — берём первый (**Id** уникален, так что всегда один), **OrDefault** — не выбрасывать исключение, а вернуть **null** если пусто.

5.3. **CreateAsync()**. Создать категорию с заметкой

```
public async Task<Category> CreateAsync(Category category) {
    _db.Categories.Add(category);
    await _db.SaveChangesAsync();
    return category;
}
```

5.4. **UpdateAsync()**. Обновить категорию с заметкой

```
public async Task<Category> UpdateAsync(Category category) {
    _db.Categories.Update(category);
    await _db.SaveChangesAsync();
    return category;
}
```

5.5. **DeleteAsync()**. Удалить категорию с заметкой.

```
public async Task DeleteAsync(Category category) {
    _db.Categories.Remove(category);
    await _db.SaveChangesAsync();
}
```

5.6. **ExistsAsync()**. Проверка существования без загрузки всего объекта

```
public async Task<bool> ExistsAsync(int id)
    => await _db.Categories.AnyAsync(c => c.Id == id);
```

5.7. **HasNotesAsync()**. Есть ли у категории заметки (для проверки перед удалением)

```
public async Task<bool> HasNotesAsync(int id)
    => await _db.Notes.AnyAsync(n => n.CategoryId == id);
```

.AnyAsync() — транслируется в *SELECT EXISTS(SELECT 1 FROM ...)*. Это самый эффективный способ проверить наличие записи: база находит первое совпадение и сразу останавливается, не загружая данные.

Сравнение подходов для проверки существования:

Подход	SQL	Загружает данные
AnyAsync(c => c.Id == id)	SELECT EXISTS(...)	Нет 
FindAsync(id) != null	SELECT * FROM ... WHERE Id = ?	Да 
CountAsync(c => c.Id == id) > 0	SELECT COUNT(*) FROM ...	Нет, но медленнее 

5.3. Интерфейс **INoteRepository**

Откройте файл **Repositories/INoteRepository.cs** и вручную перепишите код.

1. Объявляем пространства имён

```
using NotesApp.Models;
using NotesApp.Models.DTOS;
namespace NotesApp.Repositories;
```

2. Объявляем интерфейс **INoteRepository**

```
public interface INoteRepository {
    Task<IEnumerable<NoteResponseDto>> GetAllAsync(NoteFilterDto filter);
    Task<NoteResponseDto?> GetByIdAsync(int id);
    Task<Note> CreateAsync(Note note);
    Task<Note> UpdateAsync(Note note);
    Task DeleteAsync(Note note);
    Task<Note?> FindAsync(int id);
    Task<int> GetCountByCategoryAsync(int categoryId);
}
```

Откройте файл **Models/DTOs/NoteFilterDto.cs** и вручную перепишите код.

1. Объявляем пространства имён

```
namespace NotesApp.Models.DTOs;
```

2. Объявляем класс NoteFilterDto

```
public class NoteFilterDto {
    public int? CategoryId { get; set; } // фильтр по категории
    public bool? IsPinned { get; set; } // только закреплённые
    public bool Archived { get; set; } = false; // включать архивные?
    public string? Search { get; set; } // поиск по заголовку и содержанию
    public int? MinPriority { get; set; } // минимальный приоритет
    public string SortBy { get; set; } = "createdAt"; // поле сортировки
    public bool Descending { get; set; } = true; // направление
    public int Page { get; set; } = 1;
    public int PageSize { get; set; } = 10;
}
```

5.4. Реализация NoteRepository

Откройте файл **Repositories/NoteRepository.cs** и вручную перепишите код.

1. Объявляем пространства имён

```
using Microsoft.EntityFrameworkCore;
using NotesApp.Data;
using NotesApp.Models;
using NotesApp.Models.DTOs;
namespace NotesApp.Repositories;
```

2. Объявляем класс NoteRepository, который наследуется от интерфейса INoteRepository

```
public class NoteRepository : INoteRepository { }
```

3. **Внутри** класса **NoteRepository** создаём локальную базу **_db**, доступную только для чтения (**readonly**) и объявляем конструктор, который будет создавать пустую базу данных

```
private readonly AppDbContext _db;
public NoteRepository(AppDbContext db) {
    _db = db;
}
```

4. Внутри класса **NoteRepository** реализуем асинхронные методы

4.1. **GetAllAsync()**. Получить все категории с количеством заметок

```
public async Task<IEnumerable<NoteResponseDto>> GetAllAsync(NoteFilterDto filter) {
    var query = _db.Notes
        .Include(n => n.Category)
        .AsQueryable();
    // Применяем фильтры
    // Архивные заметки — по умолчанию скрываем
    query = query.Where(n => n.IsArchived == filter.Archived);
    if (filter.CategoryId.HasValue)
        query = query.Where(n => n.CategoryId == filter.CategoryId.Value);
    if (filter.IsPinned.HasValue)
        query = query.Where(n => n.IsPinned == filter.IsPinned.Value);
    if (filter.MinPriority.HasValue)
        query = query.Where(n => n.Priority >= filter.MinPriority.Value);
    if (!string.IsNullOrEmpty(filter.Search)) {
        var search = filter.Search.ToLower();
        query = query.Where(n =>
            n.Title.ToLower().Contains(search) ||
            n.Content.ToLower().Contains(search));
    }
    // Сортировка
    query = filter.SortBy.ToLower() switch {
        "title" => filter.Descending ? query.OrderByDescending(n => n.Title)
                                     : query.OrderBy(n => n.Title),
        "priority" => filter.Descending ? query.OrderByDescending(n => n.Priority)
                                       : query.OrderBy(n => n.Priority),
        "updatedat" => filter.Descending ? query.OrderByDescending(n => n.UpdatedAt)
                                        : query.OrderBy(n => n.UpdatedAt),
        _ => filter.Descending ? query.OrderByDescending(n => n.CreatedAt)
                              : query.OrderBy(n => n.CreatedAt),
    };
    // Закреплённые заметки всегда идут первыми
    query = query.OrderByDescending(n => n.IsPinned)
        .ThenBy(n => n.IsArchived);
}
```

Продолжение метода на следующей странице


```

// Пагинация
var page = Math.Max(1, filter.Page);
var pageSize = Math.Clamp(filter.PageSize, 1, 50);
return await query
    .Skip((page - 1) * pageSize)
    .Take(pageSize)
    .Select(n => new NoteResponseDto {
        Id = n.Id,
        Title = n.Title,
        Content = n.Content,
        IsPinned = n.IsPinned,
        IsArchived = n.IsArchived,
        Priority = n.Priority,
        CreatedAt = n.CreatedAt,
        UpdatedAt = n.UpdatedAt,
        CategoryId = n.CategoryId,
        CategoryName = n.Category.Name, // из JOIN'а
        CategoryColor = n.Category.Color
    })
    .ToListAsync();
}

```

AsQueryable() — получаем объект запроса, к которому можно динамически добавлять условия. На этом этапе SQL-запрос **не выполняется** — строится только его описание.

Ключевая идея: мы постепенно добавляем .Where(...) к query в зависимости от переданных фильтров. EF Core объединит все условия в один SQL-запрос, который выполнится только в конце при .ToListAsync().

```
query = query.Where(n => n.IsArchived == filter.Archived);
```

```
if (filter.CategoryId.HasValue)
```

```
    query = query.Where(n => n.CategoryId == filter.CategoryId.Value);
```

```
if (filter.IsPinned.HasValue)
```

```
    query = query.Where(n => n.IsPinned == filter.IsPinned.Value);
```

Условные фильтры. HasValue — проверяем, что nullable-свойство содержит значение (не null). .Value — достаём само значение из nullable-обёртки.

Каждое query = query.Where(...) добавляет условие AND к будущему SQL-запросу. Если CategoryId = null, строка пропускается и условие не добавляется.

```
if (!string.IsNullOrWhiteSpace(filter.Search)) {
```

```
    var search = filter.Search.ToLower();
```

```
    query = query.Where(n =>
```



```

        n.Title.ToLower().Contains(search) ||
        n.Content.ToLower().Contains(search));
    }

```

Поиск по тексту в заголовке **или** содержимом заметки.

`string.IsNullOrEmpty()` — проверяет, что строка не `null`, не пустая `""` и не состоит из пробелов `" "`. Все три случая означают «поиск не задан».

`.ToLower()` — приводим обе стороны к нижнему регистру, чтобы поиск не зависел от регистра букв: «Работа», «работа», «РАБОТА» — всё найдётся.

`||` — логическое «или»: ищем в заголовке **или** в содержимом.

```

query = filter.SortBy.ToLower() switch {
    "title"    => filter.Descending ? query.OrderByDescending(n => n.Title)
                : query.OrderBy(n => n.Title),
    "priority" => filter.Descending ? query.OrderByDescending(n => n.Priority)
                : query.OrderBy(n => n.Priority),
    "updatedat" => filter.Descending ? query.OrderByDescending(n => n.UpdatedAt)
                : query.OrderBy(n => n.UpdatedAt),
    _          => filter.Descending ? query.OrderByDescending(n => n.CreatedAt)
                : query.OrderBy(n => n.CreatedAt),
};

```

Switch expression — компактная замена `switch/case`. Значение слева от `switch` сравнивается с каждым шаблоном:

- `"title"`, `"priority"`, `"updatedat"` — конкретные строки
- `_` — шаблон «по умолчанию» (аналог `default`): если ни один вариант не подошёл, сортируем по дате создания

Тернарный оператор `condition ? A : B` — если `filter.Descending == true`, берём `OrderByDescending`, иначе `OrderBy`.

```

query = query.OrderByDescending(n => n.IsPinned)
        .ThenBy(n => n.IsArchived);

```

Двухуровневая сортировка. `.ThenBy()` — вторичная сортировка, применяется когда первичная даёт одинаковые результаты:

1. Сначала закреплённые заметки (`IsPinned = true`) — они идут первыми, потому что `true > false`
2. Среди заметок с одинаковым `IsPinned` — неархивные раньше архивных

Эта сортировка **добавляется поверх** выбранной пользователем, поэтому закреплённые всегда будут наверху вне зависимости от других настроек.

```

var page = Math.Max(1, filter.Page);
var pageSize = Math.Clamp(filter.PageSize, 1, 50);

```

Защита от некорректных значений:

- `Math.Max(1, filter.Page)` — если клиент передал `page=0` или отрицательное число, берём 1.
- `Math.Clamp(filter.PageSize, 1, 50)` — ограничиваем размер страницы диапазоном от 1 до 50.

Без этого клиент мог бы запросить `pageSize=100000` и положить сервер.

`return await query`

```
.Skip((page - 1) * pageSize)
.Take(pageSize)
.Select(n => new NoteResponseDto { ... })
.ToListAsync();
```

Пагинация через `Skip/Take`:

- `.Skip((page - 1) * pageSize)` — пропустить записи предыдущих страниц. Страница 1: пропустить 0. Страница 2: пропустить 10. Страница 3: пропустить 20.
- `.Take(pageSize)` — взять только нужное количество записей.

В SQL это транслируется в:

`ORDER BY ...`

`LIMIT 10 OFFSET 0` -- страница 1

`LIMIT 10 OFFSET 10` -- страница 2

`LIMIT 10 OFFSET 20` -- страница 3

`.Select(n => new NoteResponseDto { ... })` здесь проецирует в DTO **после** всех фильтров и пагинации

— в память загружается только финальный результат, а не все заметки.

4.2. `GetByIdAsync()`. Получить категорию без заметок (для проверки существования)

```
public async Task<NoteResponseDto?> GetByIdAsync(int id) {
    return await _db.Notes
        .Include(n => n.Category)
        .Where(n => n.Id == id)
        .Select(n => new NoteResponseDto {
            Id = n.Id,
            Title = n.Title,
            Content = n.Content,
            IsPinned = n.IsPinned,
            IsArchived = n.IsArchived,
            Priority = n.Priority,
            CreatedAt = n.CreatedAt,
            UpdatedAt = n.UpdatedAt,
            CategoryId = n.CategoryId,
            CategoryName = n.Category.Name,
            CategoryColor = n.Category.Color
        })
        .FirstOrDefaultAsync();
}
```

4.3. `CreateAsync()`. Создать категорию с заметкой

```
public async Task<Note> CreateAsync(Note note) {  
    _db.Notes.Add(note);  
    await _db.SaveChangesAsync();  
    return note;  
}
```

4.4. UpdateAsync(). Обновить категорию с заметкой

```
public async Task<Note> UpdateAsync(Note note) {  
    note.UpdatedAt = DateTime.UtcNow;  
    _db.Notes.Update(note);  
    await _db.SaveChangesAsync();  
    return note;  
}
```

note.UpdatedAt = DateTime.UtcNow; - Перед сохранением обновляем время изменения. Это делается в репозитории, а не в контроллере — централизованная логика. Теперь невозможно забыть обновить UpdatedAt при редактировании заметки из любого места кода.

.Update(note) — помечает объект как изменённый. EF Core запомнит, что нужно выполнить UPDATE.

.SaveChangesAsync() — только здесь SQL-запрос реально отправляется в базу данных. До этого все изменения хранятся в памяти в «трекере изменений» EF Core. Это позволяет сгруппировать несколько операций в одну транзакцию.

4.5. DeleteAsync(). Удалить категорию с заметкой

```
public async Task DeleteAsync(Note note) {  
    _db.Notes.Remove(note);  
    await _db.SaveChangesAsync();  
}
```

4.6. FindAsync(). Найти категорию с заметкой

```
public async Task<Note?> FindAsync(int id)  
    => await _db.Notes.FindAsync(id);
```

4.7. GetCountByCategoryAsync()

```
public async Task<int> GetCountByCategoryAsync(int categoryId)  
    => await _db.Notes.CountAsync(n => n.CategoryId == categoryId);
```

.CountAsync() — транслируется в `SELECT COUNT(*) FROM Notes WHERE CategoryId = ?`. Используется в контроллере категорий, чтобы вернуть количество заметок без их загрузки.

Практическое задание

Выполните в терминале:

dotnet build

cd ..

git add .

git commit -m "Step 5: Repository pattern - ICategoryRepository, INoteRepository and implementations"

cd NotesApp

Итоговая схема вызовов

HTTP GET /api/notes?categoryId=2&sortBy=priority&page=1

|



NotesController

| вызывает



INoteRepository.GetAllAsync(filter)

| реализован в



NoteRepository.GetAllAsync(filter)

| строит запрос динамически



_db.Notes ← начало запроса

.Include(n => n.Category) ← JOIN с Categories

.AsQueryable()

.Where(n => !n.IsArchived) ← фильтр архива

.Where(n => n.CategoryId == 2) ← фильтр категории

.OrderByDescending(n => n.Priority)

.OrderByDescending(n => n.IsPinned)

.Skip(0).Take(10) ← пагинация

.Select(n => new NoteResponseDto { ... })

.ToListAsync() ← SQL выполняется здесь

Шаг 6. Регистрация зависимостей и миграция

6.1. Program.cs

Откройте **Program.cs** и вручную перепишите код.

1. Объявляем пространства имён

```
using Microsoft.EntityFrameworkCore;  
using NotesApp.Data;  
using NotesApp.Repositories;
```

2. Создаём строитель (**builder**) приложения. Builder накапливает настройки: какие сервисы использовать, как они связаны между собой. Пока приложение ещё не запущено — мы только его конфигурируем.

```
var builder = WebApplication.CreateBuilder(args);
```

3. Регистрируем поддержку контроллеров **AddControllers()**. Без этой строки ASP.NET Core не будет знать о файлах в папке Controllers/ и не будет обрабатывать маршруты вида [HttpGet], [HttpPost].

```
builder.Services.AddControllers();  
builder.Services.AddEndpointsApiExplorer();  
builder.Services.AddSwaggerGen(c => {  
    c.SwaggerDoc("v1", new() {  
        Title = "NotesApp API",  
        Version = "v1",  
        Description = "API для управления заметками по категориям"  
    });  
});
```

Настройка **Swagger** — инструмента для автоматической документации и тестирования API.

- **AddEndpointsApiExplorer()** — сканирует все маршруты приложения, чтобы Swagger знал, что документировать.
- **AddSwaggerGen()** — настраивает генератор документации. **SwaggerDoc("v1", ...)** создаёт документ версии v1 с заголовком и описанием. Это то, что вы видите вверху страницы Swagger UI.

4. Добавляем регистрацию **AppDbContext** в DI-контейнере.

```
builder.Services.AddDbContext<AppDbContext>(options =>  
    options.UseSqlite(builder.Configuration.GetConnectionString("NotesDb"));
```

- **AddDbContext<AppDbContext>** — говорим DI: «когда кто-то запросит AppDbContext, создай его».

- **options.UseSqlite(...)** — указываем, какую базу данных использовать (SQLite).
- **builder.Configuration.GetConnectionString("NotesDb")** — читает строку подключения из appsettings.json по ключу "NotesDb". Вместо того чтобы хранить путь к файлу БД прямо в коде, мы храним его в конфигурационном файле.

5. Регистрация репозитория

```
builder.Services.AddScoped<ICategoryRepository, CategoryRepository>();
builder.Services.AddScoped<INoteRepository, NoteRepository>();
```

Синтаксис **AddScoped<Интерфейс, Реализация>()** говорит DI-контейнеру:

«Когда кто-то попросит **ICategoryRepository** — дай ему объект

CategoryRepository»

Именно поэтому в контроллере можно написать:

```
public CategoriesController(ICategoryRepository repo)
```

...и ASP.NET Core автоматически создаст **CategoryRepository** и передаст его.

AddScoped — один объект на HTTP-запрос. Это стандартный выбор для репозитория и DbContext: все операции в рамках одного запроса используют одно и то же соединение с БД, что позволяет объединять их в транзакции.

6. CORS

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowAll", p =>
        p.AllowAnyOrigin().AllowAnyMethod().AllowAnyHeader());
});
```

CORS (Cross-Origin Resource Sharing) — механизм безопасности браузера. По умолчанию браузер блокирует запросы с одного домена (localhost:3000) к другому (localhost:5000). CORS-политика "AllowAll" снимает это ограничение для всех источников.

- **AllowAnyOrigin()** — принимаем запросы с любого домена
- **AllowAnyMethod()** — разрешаем GET, POST, PUT, DELETE и другие методы
- **AllowAnyHeader()** — принимаем любые заголовки

Политику именуем **"AllowAll"** — это имя используется ниже в **app.UseCors("AllowAll")**.

7. Приложение

```
var app = builder.Build();
```

Финализируем конфигурацию и создаём объект приложения.

8. Автоматическое применение миграций при старте

```
using (var scope = app.Services.CreateScope()) {  
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();  
    db.Database.Migrate();  
}
```

using (var scope = ...) — создаём временную «область» DI-контейнера. Это нужно, потому что **AppDbContext** зарегистрирован как **Scoped** (живёт в рамках HTTP-запроса), а здесь мы вне HTTP-запроса. **using** гарантирует, что по завершении блока **scope** будет уничтожен вместе с **DbContext**.

scope.ServiceProvider.GetRequiredService<AppDbContext>() — вручную запрашиваем **AppDbContext** из DI-контейнера. Если по какой-то причине он не зарегистрирован — выброс исключения (это лучше, чем молчаливое **null**).

db.Database.Migrate() — проверяет, какие миграции уже применены, и применяет все недостающие. Эквивалент команды **dotnet ef database update**, но выполняется автоматически при каждом запуске.

9. Swagger

```
if (app.Environment.IsDevelopment()) {  
    app.UseSwagger();  
    app.UseSwaggerUI();  
}
```

10. Конвейер middleware

```
app.UseHttpsRedirection();  
app.UseCors("AllowAll");  
app.UseAuthorization();  
app.MapControllers();
```

Цепочка обработчиков, через которые проходит каждый HTTP-запрос по порядку:

Строка	Что делает
UseHttpsRedirection()	Перенаправляет HTTP → HTTPS
UseCors("AllowAll")	Добавляет заголовки CORS в ответ (применяет нашу политику)
UseAuthorization()	Проверяет права доступа (пока не настроено, но обязательно присутствует)
MapControllers()	Подключает маршруты из всех контроллеров
Run()	Запускает сервер и начинает слушать входящие запросы

Порядок строк важен: **UseCors** должен стоять до **UseAuthorization**. Если поставить их наоборот, CORS-заголовки не будут добавлены к ответам с ошибкой авторизации.

11. Запускаем приложение

```
app.Run();
```

Обратите внимание на блок автоматического применения миграций:

```
using (var scope = app.Services.CreateScope()) {  
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();  
    db.Database.Migrate();  
}
```

6.2. Создание и применение миграции

Выполните в терминале:

```
dotnet ef migrations add InitialCreate
```

```
dotnet ef database update
```

Убедитесь, что файл **notesapp.db** появился в папке **NotesApp**.

 **notesapp.db**

Откройте его в **SQLite Viewer** и проверьте:

- Таблица **Categories** с тремя записями
- Таблица **Notes** с пятью записями
- Колонка **CategoryId** в таблице **Notes**

Практическое задание

1. Выполните **dotnet ef migrations list**
2. Сделайте скриншот SQLite Viewer с двумя таблицами
3. Сохраните в **img/step6_migrationLab33-34_FIO.png**

Выполните в терминале:

```
cd ..
```

```
git add .
```

```
git commit -m "Step 6: DI registered, migrations applied, database created"
```

```
cd NotesApp
```

Шаг 7. Контроллер категорий

Откройте файл **Controllers/CategoriesController.cs** и вручную перепишите код.

1. Объявляем пространства имён

```
using Microsoft.AspNetCore.Mvc;
using NotesApp.Helpers;
using NotesApp.Models;
using NotesApp.Models.DTOs;
using NotesApp.Repositories;
namespace NotesApp.Controllers;
```

2. Объявляем класс CategoriesController

```
[ApiController]
[Route("api/[controller]")]
public class CategoriesController : ControllerBase { }
```

3. Внутри класса CategoriesController объявляем поле и конструктор

```
private readonly ICategoryRepository _repo;
public CategoriesController(ICategoryRepository repo) {
    _repo = repo;
}
```

DI-контейнер автоматически создаст объект **CategoryRepository** и передаст его сюда при каждом HTTP-запросе. Контроллер работает через интерфейс **ICategoryRepository**, а не напрямую с классом — он не знает и не должен знать, как именно устроена работа с базой данных.

4. Внутри класса CategoriesController объявляем маршруты

4.1. GET /api/categories

```
[HttpGet]
public async
Task<ActionResult<ApiResponse<IEnumerable<CategoryResponseDto>>>>
GetAll() {
    var categories = await _repo.GetAllAsync();
    return Ok(ApiResponse<IEnumerable<CategoryResponseDto>>.Ok(
categories));
}
```

Тип возвращаемого значения выглядит сложно, но разберём послойно:

Слой	Зачем
ActionResult<...>	Позволяет вернуть как данные, так и HTTP-статус (200, 404, 400)
ApiResponse<...>	Наша обёртка: { success, data, message, statusCode }
IEnumerable<CategoryResponseDto>	Коллекция DTO категорий — сами данные

Здесь два разных Ok:

- Внешний Ok(...) — метод ControllerBase, устанавливает HTTP-статус 200 и сериализует аргумент в JSON
- Внутренний .Ok(categories) — наш статический метод ApiResponse<T>.Ok(), создающий объект-обёртку с полями success: true, data: [...]

Клиент получит:

```
{
  "success": true,
  "message": "Успешно",
  "data": [ { "id": 1, "name": "Работа", ... } ],
  "statusCode": 200
}
```

4.2. GET /api/categories/id

```
[HttpGet("{id}")]
public async Task<ActionResult<ApiResponse<Category>>> GetById(int id) {
    var category = await _repo.GetByIdAsync(id);
    if (category is null)
        return NotFound(ApiError.NotFound($"Категория с id={id} не найдена"));
    return Ok(ApiResponse<Category>.Ok(category));
}
```

{id} — параметр маршрута. ASP.NET Core автоматически извлечёт значение из URL и передаст в параметр метода int id. Запрос GET /api/categories/5 → id = 5.

if (category is null)

return NotFound(ApiError.NotFound(\$"Категория с id={id} не найдена"));

is null — современный способ проверки на null в C# (эквивалент == null, но компилятор предпочитает is null для явной проверки ссылок).

`return NotFound(...)` — метод `ControllerBase`, устанавливает HTTP-статус 404 и сериализует `ApiError` в JSON.

4.3. GET /api/categories/id/notes

```
[HttpGet("{id}/notes")]
public async Task<ActionResult> GetWithNotes(int id) {
    var category = await _repo.GetByIdWithNotesAsync(id);
    if (category is null)
        return NotFound(ApiError.NotFound($"Категория с id={id} не найдена"));
    // Формируем ответ: категория + её неархивные заметки
    var response = new {
        Id = category.Id,
        Name = category.Name,
        Description = category.Description,
        Color = category.Color,
        NotesCount = category.Notes.Count,
        Notes = category.Notes.Select(n => new {
            n.Id,
            n.Title,
            n.Priority,
            n.IsPinned,
            n.CreatedAt
        })
    };
    return Ok(ApiResponse<object>.Ok(response));
}
```

`response` - Анонимный тип — объект без заранее объявленного класса. C# сам вычислит его тип на этапе компиляции. Удобен для одноразовых ответов, где создавать отдельный DTO-класс избыточно.

`.Select()` проецирует каждую заметку в анонимный объект только с нужными полями. Запись `n.Id` внутри инициализатора анонимного типа — сокращение от `Id = n.Id` (C# сам берёт имя свойства).

Мы намеренно не включаем `n.Content` — в списке нужны только заголовки, контент загружается при открытии конкретной заметки.

`ApiResponse<object>` — используем `object` как тип, потому что анонимный тип нельзя указать явно. При сериализации в JSON это не имеет значения: `System.Text.Json` сериализует все публичные свойства объекта независимо от его типа.

4.4. POST /api/categories

```

[HttpPost]
public async Task<ActionResult<ApiResponse<Category>>> Create([FromBody]
CreateCategoryDto dto) {
    // [ApiController] автоматически вернёт 400 если модель невалидна
    // Но мы можем добавить дополнительные проверки вручную
    if (!ModelState.IsValid) {
        var errors = ModelState.Values
            .SelectMany(v => v.Errors)
            .Select(e => e.ErrorMessage)
            .ToList();
        return BadRequest(ApiError.BadRequest("Ошибки валидации", errors));
    }
    var category = new Category {
        Name = dto.Name.Trim(),
        Description = dto.Description.Trim(),
        Color = dto.Color,
        CreatedAt = DateTime.UtcNow
    };
    var created = await _repo.CreateAsync(category);
    return CreatedAtAction(
        nameof(GetById),
        new { id = created.Id },
        ApiResponse<Category>.Created(created, "Категория успешно создана"));
}

```

[FromBody] — указывает ASP.NET Core читать параметр из тела HTTP-запроса (JSON). Без этого атрибута фреймворк попытался бы искать параметры в URL-строке. При наличии [ApiController] на классе [FromBody] добавляется автоматически, но явное указание делает код понятнее.

ModelState — словарь с результатами валидации. ASP.NET Core автоматически проверяет атрибуты [Required], [MaxLength], [RegularExpression] из DTO и записывает ошибки в ModelState.

Разбор цепочки LINQ:

- ModelState.Values — коллекция результатов по каждому полю
- .SelectMany(v => v.Errors) — разворачиваем вложенные коллекции ошибок в один плоский список (SelectMany — аналог «flatten»)
- .Select(e => e.ErrorMessage) — берём только текст ошибки
- .ToList() — материализуем в List<string>

Примечание: [ApiController] уже возвращает 400 автоматически — явная проверка ModelState здесь избыточна, но показывает, как извлечь ошибки в нашем формате ApiResponse вместо стандартного формата ASP.NET Core.

CreatedAtAction — специальный метод ControllerBase для ответа 201 Created. Он делает три вещи:

- Устанавливает HTTP-статус 201
- Добавляет в заголовок ответа Location: /api/categories/7 — адрес созданного ресурса
- Сериализует тело ответа

Аргументы:

- nameof(GetById) — имя метода, который возвращает созданный ресурс. nameof — оператор, возвращающий имя идентификатора как строку: "GetById". Лучше, чем строка в кавычках: при переименовании метода компилятор выдаст ошибку.
- new { id = created.Id } — параметры маршрута для генерации URL.
- Третий аргумент — тело ответа.

4.5. PUT /api/categories/5

```
[HttpPut("{id}")]
public async Task<ActionResult<ApiResponse<Category>>> Update(
    int id, [FromBody] UpdateCategoryDto dto) {
    var category = await _repo.GetByIdAsync(id);
    if (category is null)
        return NotFound(ApiError.NotFound($"Категория с id={id} не найдена"));
    category.Name = dto.Name.Trim();
    category.Description = dto.Description.Trim();
    category.Color = dto.Color;
    var updated = await _repo.UpdateAsync(category);
    return Ok(ApiResponse<Category>.Ok(updated, "Категория обновлена"));
}
```

Паттерн «найди и обнови»:

1. Загружаем существующий объект из БД
2. Если не нашли — возвращаем 404
3. Изменяем только те поля, которые пришли от клиента
4. Сохраняем

Это важно: мы не создаём новый объект Category, а меняем загруженный. EF Core отслеживает изменения и знает, какие именно поля обновить в SQL. Поле CreatedAt остаётся нетронутым — клиент не может изменить дату создания.

4.6. DELETE /api/categories/5

```
[HttpDelete("{id}")]
public async Task<ActionResult> Delete(int id) {
    var category = await _repo.GetByIdAsync(id);
    if (category is null)
        return NotFound(ApiError.NotFound($"Категория с id={id} не найдена"));
    // Проверяем, есть ли заметки в категории
    if (await _repo.HasNotesAsync(id)) {
        return BadRequest(ApiError.BadRequest(
            "Невозможно удалить категорию: в ней есть заметки. " +
            "Сначала удалите или переместите заметки."));
    }
    await _repo.DeleteAsync(category);
    return NoContent();
}
```

Проверяем наличие заметок до удаления. Хотя `OnDelete(DeleteBehavior.Restrict)` в `DbContext` всё равно не позволит удалить категорию с заметками, без этой проверки клиент получил бы техническое исключение от БД с непонятным сообщением. Явная проверка даёт понятный ответ 400 с человекочитаемым текстом.

`NoContent()` — метод `ControllerBase`, возвращает 204 No Content. По стандарту REST, DELETE успешно выполненный возвращает 204 — операция прошла, но возвращать нечего (объект удалён). Это отличает успешное удаление (204) от «не нашли» (404).

Итоговая таблица маршрутов контроллера

Метод	Маршрут	Статус успеха	Статус ошибки
GetAll	GET /api/categories	200 OK	—
GetById	GET /api/categories/{id}	200 OK	404
GetWithNotes	GET /api/categories/{id}/notes	200 OK	404
Create	POST /api/categories	201 Created	400
Update	PUT /api/categories/{id}	200 OK	404
Delete	DELETE /api/categories/{id}	204 No Content	404, 400

Практическое задание

1. Соберите и запустите:

dotnet build

dotnet run

2. Откройте **Swagger UI** и протестируйте контроллер категорий:

- **GET /api/categories** — список всех категорий с количеством заметок
- **GET /api/categories/2/notes** — категория «Учёба» с заметками

- **POST** — создайте новую категорию с невалидным цветом (например **"color": "red"**) — должна вернуть ошибка 400 с описанием
- **POST** — создайте категорию с правильным HEX-цветом — должна вернуть 201
- **DELETE /api/categories/1** — должна вернуть 400 (в категории есть заметки)

3. Обратите внимание на единый формат ответов: поля **success, data, message** везде одинаковые

4. Сделайте скриншоты: успешный GET и попытка удаления категории с заметками

5. Сохраните в **img/step7_categoriesLab33-34_FIO.png**

Выполните в терминале:

cd ..

git add .

git commit -m "Step 7: CategoriesController with ApiResponse wrapper"

cd NotesApp

Шаг 8. Контроллер заметок

Откройте файл **Controllers/NotesController.cs** и вручную перепишите код.

1. Объявляем пространства имён

```
using Microsoft.AspNetCore.Mvc;
using NotesApp.Helpers;
using NotesApp.Models;
using NotesApp.Models.DTOS;
using NotesApp.Repositories;
namespace NotesApp.Controllers;
```

2. Объявляем класс **NotesController**

```
[ApiController]
[Route("api/[controller]")]
public class NotesController : ControllerBase { }
```

3. Внутри класса **NotesController** объявляем поля и конструктор:

```
private readonly INoteRepository _noteRepo;
private readonly ICategoryRepository _categoryRepo;

public NotesController(INoteRepository noteRepo,
ICategoryRepository categoryRepo) {
    _noteRepo = noteRepo;
    _categoryRepo = categoryRepo;
}
```

Контроллер заметок зависит от **двух** репозиториев. **_categoryRepo** нужен для проверки: существует ли категория, которую указывает клиент при создании или обновлении заметки. DI-контейнер автоматически создаст оба объекта и передаст их через конструктор — мы просто перечисляем нужные зависимости, а фреймворк делает остальное.

4. Внутри класса **NotesController** создаём маршруты контроллера

4.1. GET /api/notes

```
[HttpGet]
public async Task<ActionResult<ApiResponse<IEnumerable<NoteResponseDto>>>> GetAll(
    [FromQuery] int? categoryId = null,
    [FromQuery] bool? isPinned = null,
    [FromQuery] bool archived = false,
    [FromQuery] string? search = null,
    [FromQuery] int? minPriority = null,
    [FromQuery] string sortBy = "createdAt",
    [FromQuery] bool descending = true,
    [FromQuery] int page = 1,
    [FromQuery] int pageSize = 10) {
    var filter = new NoteFilterDto {
        CategoryId = categoryId,
        IsPinned = isPinned,
        Archived = archived,
        Search = search,
        MinPriority = minPriority,
        SortBy = sortBy,
        Descending = descending,
        Page = page,
        PageSize = pageSize
    };
    var notes = await _noteRepo.GetAllAsync(filter);
    return Ok(ApiResponse<IEnumerable<NoteResponseDto>>.Ok(notes));
}
```

[FromQuery] — указывает ASP.NET Core читать параметр из строки URL-запроса (query string), а не из тела.

ASP.NET Core автоматически разберёт строку запроса и передаст значения в соответствующие параметры метода.

Все параметры имеют значения по умолчанию — ни один не является обязательным. Клиент может передать любую комбинацию или вообще ничего: **GET /api/notes** вернёт все неархивные заметки с настройками по умолчанию.

Nullable-типы (**int?**, **bool?**, **string?**) позволяют различить «параметр не передан» (**null**) и «передан конкретный ноль/false». Без **?** невозможно узнать: клиент явно написал **categoryId=0** или просто не передал параметр.

Собираем все параметры в объект **NoteFilterDto** и передаём его в репозиторий. Это чище, чем передавать 9 параметров напрямую в метод репозитория — и проще расширять: добавление нового фильтра требует изменения только **NoteFilterDto**, а не сигнатуры метода репозитория.

4.2. GET /api/notes/id

```
[HttpGet("{id}")]
public async Task<ActionResult<ApiResponse<NoteResponseDto>>> GetById(int id) {
    var note = await _noteRepo.GetByIdAsync(id);
    if (note is null)
        return NotFound(ApiError.NotFound($"Заметка с id={id} не найдена"));
    return Ok(ApiResponse<NoteResponseDto>.Ok(note));
}
```

Используем **_noteRepo.GetByIdAsync(id)** — метод репозитория, который выполняет JOIN с таблицей **Categories** и возвращает **NoteResponseDto** с полями **CategoryName** и **CategoryColor**. Клиент получает всю нужную информацию в одном запросе, без дополнительных обращений к API.

4.3. POST /api/notes

```
[HttpPost]
public async Task<ActionResult<ApiResponse<NoteResponseDto>>> Create(
    [FromBody] CreateNoteDto dto) {
    // Проверяем, существует ли категория
    if (!await _categoryRepo.ExistsAsync(dto.CategoryId))
        return BadRequest(ApiError.BadRequest(
            "Ошибки валидации",
            new List<string> { $"Категория с id={dto.CategoryId} не существует" }));
    var note = new Note {
        Title = dto.Title.Trim(),
        Content = dto.Content.Trim(),
        Priority = dto.Priority,
        CategoryId = dto.CategoryId,
        IsPinned = false,
        IsArchived = false,
        CreatedAt = DateTime.UtcNow,
        UpdatedAt = DateTime.UtcNow
    };
    var created = await _noteRepo.CreateAsync(note);
    // Загружаем полный ответ с данными категории
    var response = await _noteRepo.GetByIdAsync(created.Id);
    return CreatedAtAction(
        nameof(GetById),
        new { id = created.Id },
        ApiResponse<NoteResponseDto>.Created(response!, "Заметка успешно создана"));
}
```

Кросс-валидация — проверка согласованности данных между двумя таблицами. Атрибуты **[Required]** и **[Range]** в DTO проверяют только формат данных, но не могут проверить, существует ли запись с таким Id в другой таблице.

!await — сначала выполняется **await _categoryRepo.ExistsAsync(...)** (возвращает bool), затем **!** инвертирует результат. Читается как: «если категория НЕ существует».

Без этой проверки при несуществующем **categoryId** база данных выбросила бы исключение.

Служебные поля **IsPinned**, **IsArchived**, **CreatedAt**, **UpdatedAt** устанавливаются сервером явно, а не берутся из **DTO**. Клиент не может создать заметку уже закреплённой или с произвольной датой создания — это намеренное ограничение.

После сохранения заметки выполняется второй запрос — **GetByIdAsync**. Зачем?

CreateAsync возвращает объект **Note** — модель без данных категории (**CategoryName**, **CategoryColor** отсутствуют). Чтобы вернуть клиенту полный **NoteResponseDto** с именем и цветом категории, нужно загрузить заметку повторно — уже с JOIN.

response! — оператор **null-forgiving**. Мы только что создали заметку и знаем, что она существует в БД. Компилятор этого не знает (метод возвращает **NoteResponseDto?**), поэтому **!** говорит: «этот **null** невозможен».

4.4. PUT /api/notes/id

```
[HttpPut("{id}")]
public async Task<ActionResult<ApiResponse<NoteResponseDto>>>
Update(
    int id, [FromBody] UpdateNoteDto dto) {
    var note = await _noteRepo.FindAsync(id);
    if (note is null)
        return NotFound(ApiError.NotFound($"Заметка с id={id}
не найдена"));
    // Проверяем, существует ли новая категория
    if (!await _categoryRepo.ExistsAsync(dto.CategoryId))
        return BadRequest(ApiError.BadRequest(
            "Ошибки валидации",
            new List<string> { $"Категория с id={dto.
CategoryId} не существует" }));
    note.Title = dto.Title.Trim();
    note.Content = dto.Content.Trim();
    note.Priority = dto.Priority;
    note.CategoryId = dto.CategoryId;
    await _noteRepo.UpdateAsync(note);
    var response = await _noteRepo.GetByIdAsync(id);
    return Ok(ApiResponse<NoteResponseDto>.Ok(response!,
"Заметка обновлена"));
}
```

Здесь используется **FindAsync** (из репозитория), а не **GetByIdAsync**. Отличие принципиальное:

Метод	Возвращает	Когда использовать
FindAsync(id)	Note (модель EF Core)	Нужно изменить объект и сохранить
GetByIdAsync(id)	NoteResponseDto (DTO)	Нужно только прочитать и вернуть клиенту

Для обновления нужен именно объект **Note**, который EF Core отслеживает — тогда **.Update()** сгенерирует правильный **UPDATE**-запрос.

Обновляем только поля из **UpdateNoteDto**. Поля **IsPinned**, **IsArchived**, **CreatedAt** не трогаем — они управляются отдельными маршрутами или вообще не меняются. **UpdatedAt** обновится автоматически внутри **_noteRepo.UpdateAsync(note)**

4.5. PATCH /api/notes/id/pin

```
[HttpPatch("{id}/pin")]
public async Task<ActionResult<ApiResponse<NoteResponseDto>>>
TogglePin(int id) {
    var note = await _noteRepo.FindAsync(id);
    if (note is null)
        return NotFound(ApiError.NotFound($"Заметка с id={id}
не найдена"));
    note.IsPinned = !note.IsPinned;
    await _noteRepo.UpdateAsync(note);
    var response = await _noteRepo.GetByIdAsync(id);
    var message = note.IsPinned ? "Заметка закреплена" :
"Заметка откреплена";
    return Ok(ApiResponse<NoteResponseDto>.Ok(response!,
message));
}
```

PATCH — HTTP-метод для **частичного** обновления ресурса. Отличается от PUT:

- **PUT** заменяет весь ресурс (все поля)
- **PATCH** изменяет только одно конкретное свойство

Маршрут **/api/notes/id/pin** — вложенный маршрут, описывает действие над конкретным свойством заметки. Это RESTful-стиль для операций-переключателей.

```
note.IsPinned = !note.IsPinned;
```

Переключатель (toggle): инвертируем текущее значение. Если true → становится false, если false → становится true. Один маршрут обрабатывает оба направления — и закрепление, и открепление.

```
var message = note.IsPinned ? "Заметка закреплена" : "Заметка откреплена";
```

Тернарный оператор формирует сообщение **после** обновления, когда **note.IsPinned** уже содержит новое значение. Клиент получит понятный текст о том, что произошло.

4.6. PATCH /api/notes/id/archive

```
[HttpPatch("{id}/archive")]
public async Task<ActionResult<ApiResponse<NoteResponseDto>>>
ToggleArchive(int id) {
    var note = await _noteRepo.FindAsync(id);
    if (note is null)
        return NotFound(ApiError.NotFound($"Заметка с id={id} не
найдена"));
    note.IsArchived = !note.IsArchived;
    // Архивные заметки не могут быть закреплены
    if (note.IsArchived)
        note.IsPinned = false;
    await _noteRepo.UpdateAsync(note);
    var response = await _noteRepo.GetByIdAsync(id);
    var message = note.IsArchived ? "Заметка архивирована" : "Заметка
восстановлена из архива";
    return Ok(ApiResponse<NoteResponseDto>.Ok(response!, message));
}
```

Бизнес-правило: архивная заметка не может быть закреплённой. Это логично с точки зрения UX — закреплённые заметки показываются вверху активного списка, а архивные в него не входят.

Когда **IsArchived** переключается в **true** — принудительно сбрасываем **IsPinned = false**. В обратную сторону (при восстановлении из архива) **IsPinned** не трогаем — пользователь сам решит, закреплять ли заметку снова.

Оба изменения сохраняются одним вызовом **UpdateAsync** — EF Core обновит обе колонки в одном **UPDATE**-запросе.

4.7. DELETE /api/notes/id

```
[HttpDelete("{id}")]
public async Task<ActionResult> Delete(int id) {
    var note = await _noteRepo.FindAsync(id);
    if (note is null)
        return NotFound(ApiError.NotFound($"Заметка с id={id} не найдена"));
    await _noteRepo.DeleteAsync(note);
    return NoContent();
}
```

Удаление заметки проще удаления категории: не нужно проверять связанные записи, потому что у заметки нет дочерних сущностей.

Итоговая таблица маршрутов контроллера

Метод	Маршрут	Что делает	Статус успеха	Статус ошибки
GetAll	GET /api/notes	Список заметок с фильтрами	200	—
GetById	GET /api/notes/{id}	Одна заметка	200	404
Create	POST /api/notes	Создать заметку	201	400
Update	PUT /api/notes/{id}	Обновить заметку	200	404, 400
TogglePin	PATCH /api/notes/{id}/pin	Закрепить / открепить	200	404
ToggleArchive	PATCH /api/notes/{id}/archive	Архивировать / восстановить	200	404
Delete	DELETE /api/notes/{id}	Удалить заметку	204	404

Как всё связано: полный путь запроса

GET /api/notes?categoryId=2&search=LINQ&page=1

|

▼

[FromQuery] параметры → собираются в NoteFilterDto

|

▼

NotesController.GetAll(filter)

|

вызывает

▼

INoteRepository.GetAllAsync(filter)

|

реализован в NoteRepository

▼

_db.Notes

```
.Include(n => n.Category)      ← JOIN Categories
.Where(n => !n.IsArchived)     ← только активные
.Where(n => n.CategoryId == 2) ← фильтр по категории
.Where(n => Title/Content содержит "LINQ")
.OrderByDescending(n => n.IsPinned) ← закреплённые первыми
.Skip(0).Take(10)             ← страница 1
.Select(n => new NoteResponseDto { ← проекция в DTO
    CategoryName = n.Category.Name, ← данные из JOIN
    CategoryColor = n.Category.Color
})
.ToListAsync()                ← SQL выполняется здесь
```

|

▼

ApiResponse<IEnumerable<NoteResponseDto>>.Ok(notes)

|

▼

JSON-ответ клиенту:

```
{
  "success": true,
  "message": "Успешно",
  "data": [
    {
      "id": 4,
      "title": "Конспект по LINQ",
      "categoryName": "Учёба",
      "categoryColor": "#3498db",
      ...
    }
  ],
  "statusCode": 200
}
```

Практическое задание

1. Запустите проект и протестируйте контроллер заметок в Swagger:

- **GET /api/notes** — все неархивные заметки с данными категорий
- **GET /api/notes?categoryId=2** — заметки только из категории «Учёба»
- **GET /api/notes?search=LINQ&minPriority=3** — поиск с фильтром по приоритету
- **POST /api/notes** — создайте заметку с несуществующим categoryId=99 → ожидаем 400
- **POST /api/notes** — создайте заметку с существующей категорией → ожидаем 201
- **PATCH /api/notes/1/pin** — закрепите/открепите заметку
- **PATCH /api/notes/5/archive** — архивируйте заметку
- **GET /api/notes?archived=true** — убедитесь, что архивная заметка появляется только с этим параметром

2. Обратите внимание: в ответе каждой заметки есть поля **categoryName** и **categoryColor** — они пришли из **JOIN** с таблицей **Categories**, а не из отдельного запроса.

3. Создайте файл **requests.http** в папке **NotesApp** и добавьте туда запросы для тестирования:

```
@baseUrl = http://localhost:5000
### Все заметки
Send Request
GET {{baseUrl}}/api/notes
### Заметки категории "Учёба" (id=2)
Send Request
GET {{baseUrl}}/api/notes?categoryId=2
### Поиск по слову "LINQ"
Send Request
GET {{baseUrl}}/api/notes?search=LINQ
### Только закреплённые, высокий приоритет
Send Request
GET {{baseUrl}}/api/notes?isPinned=true&minPriority=4
### Архивные заметки
Send Request
GET {{baseUrl}}/api/notes?archived=true
### Все категории с количеством заметок
Send Request
GET {{baseUrl}}/api/categories
### Категория "Учёба" со списком заметок
Send Request
GET {{baseUrl}}/api/categories/2/notes
### Создать заметку
Send Request
POST {{baseUrl}}/api/notes
Content-Type: application/json
{
  "title": "Заметка из REST Client",
  "content": "Тестируем полноценный CRUD с паттерном Repository",
  "priority": 4,
  "categoryId": 2
}
### Создать категорию
Send Request
POST {{baseUrl}}/api/categories
Content-Type: application/json
{
  "name": "Хобби",
  "description": "Заметки о хобби и увлечениях",
  "color": "■ #9b59b6"
}
### Закрепить заметку
Send Request
PATCH {{baseUrl}}/api/notes/3/pin
### Архивировать заметку
Send Request
PATCH {{baseUrl}}/api/notes/3/archive
```

4. Сделайте скриншоты: **успешный GET** с данными категории в ответе и **POST** с несуществующей категорией

5. Сохраните в **img/step8_notesLab33-34_FIO.png**

Выполните в терминале:

cd ..

git add .

git commit -m "Step 8: NotesController with full CRUD and filters"

cd NotesApp

git push

Самостоятельное задание

Задание. README.md

Создайте подробный **README.md**:

1. Заголовок и информация о студенте
2. Описание проекта (что это за приложение)
3. Структура проекта (дерево файлов)
4. Таблица всех маршрутов (метод, URL, описание, статусы ответа)
5. Описание паттерна Repository и зачем он нужен

Обязательная таблица маршрутов:

Метод	URL	Описание	Коды ответа
GET	/api/categories	Все категории с кол-вом заметок	200
GET	/api/categories/{id}	Одна категория	200, 404
GET	/api/categories/{id}/notes	Категория с заметками	200, 404
POST	/api/categories	Создать категорию	201, 400
PUT	/api/categories/{id}	Обновить категорию	200, 400, 404
DELETE	/api/categories/{id}	Удалить категорию	204, 400, 404
GET	/api/notes	Все заметки с фильтрами	200
GET	/api/notes/{id}	Одна заметка	200, 404
POST	/api/notes	Создать заметку	201, 400
PUT	/api/notes/{id}	Обновить заметку	200, 400, 404
PATCH	/api/notes/{id}/pin	Закрепить / открепить	200, 404
PATCH	/api/notes/{id}/archive	Архивировать / восстановить	200, 404
DELETE	/api/notes/{id}	Удалить заметку	204, 404

Главные выводы:

1. Паттерн **Repository** — не лишняя абстракция, а способ держать код управляемым при росте проекта.
2. **Data Annotations** решают двойную задачу: валидируют входные данные и задают ограничения в БД.

3. Единый формат ответа (**ApiResponse**) упрощает жизнь фронтенду — он всегда знает, что ждать от сервера.
4. **DeleteBehavior.Restrict** защищает данные от случайного каскадного удаления.
5. **Include()** и проекция в **DTO** через **Select()** — правильный способ получить связанные данные без N+1 проблемы.

Выполните в терминале:

cd ..

git add .

git commit -m "Added complete README for Lab33-34"

git push

Итоговая таблица: что изучили в лабораторных

Концепция / инструмент	Описание
Паттерн Repository	Изоляция логики доступа к данным от логики контроллера
Интерфейс (interface)	Контракт: что умеет репозиторий, без деталей реализации
AddScoped<I, T>()	Регистрация зависимости: один объект на HTTP-запрос
Связь один-ко-многим	HasOne(...).WithMany(...).HasForeignKey(...)
Навигационное свойство	public List<Note> Notes — ссылка на связанные объекты
Include(c => c.Notes)	Загрузка связанных данных — LEFT JOIN в SQL
DeleteBehavior.Restrict	Запрет удаления родителя при наличии дочерних записей
HasIndex(...)	Индекс для ускорения запросов по часто фильтруемому полю
Select(n => new {...})	Проекция в DTO прямо в SQL-запросе — без лишних данных
AnyAsync(...)	SELECT EXISTS(...) — быстрая проверка без загрузки объекта
ApiResponse<T>	Единый формат ответов: success, data, message
Data Annotations	[Required], [MaxLength], [Range], [RegularExpression]
[RegularExpression]	Валидация по шаблону — проверка HEX-цвета #RRGGBB
db.Database.Migrate()	Автоматическое применение миграций при старте приложения
Math.Clamp(v, min, max)	Ограничение значения в диапазоне — удобно для пагинации
null!	Null-forgiving operator для навигационных свойств
UpdatedAt	Автоматическое обновление времени изменения в репозитории